

Tutorial 15

Long Short-Term Memory (LSTM)

TASK 1

Dataset

- Google (GOOGL) historical stock price data was fetched from Yahoo Finance.
- Date range: January 1, 2010 to December 1, 2024
- Total data points: 3,753 trading days
- Feature used: Closing price only

Preprocessing

- Data was scaled to the range [0, 1] using MinMaxScaler.
- An 80/20 train-test split was applied: 2,942 training samples, remaining for testing.
- Sequences of 60 days were used as input to predict the next day's closing price.
- X_train shape: (2942, 60, 1) , y_train shape: (2942, 1)

Model Architecture

- A stacked LSTM model was implemented in PyTorch with the following structure:
 - LSTM Layer 1: 1 input → 50 hidden units, return_sequences=True
 - LSTM Layer 2: 50 hidden units, return_sequences=False, Dropout(0.2)
 - Dense Layer 1: Linear(50 → 25) with ReLU activation
 - Dense Layer 2: Linear(25 → 1) — final price output
- Device: CUDA (GPU-accelerated training)

Training Setup

- Loss Function: Mean Squared Error (MSE)
- Optimizer: Adam
- Batch Size: 32
- Epochs: 10

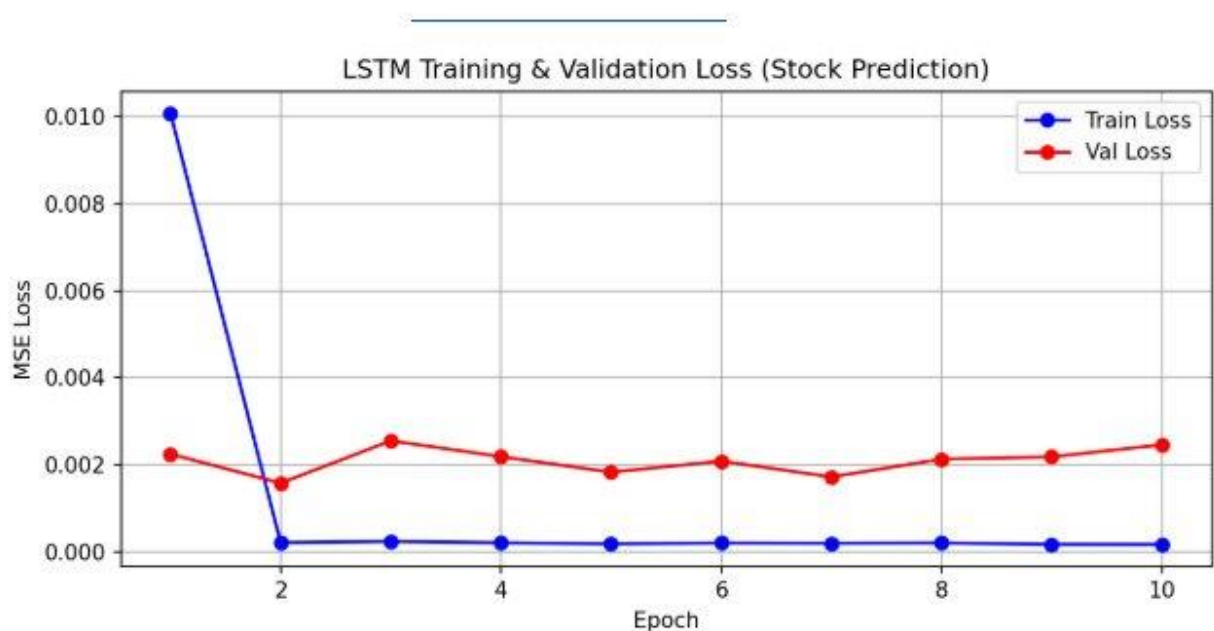
```

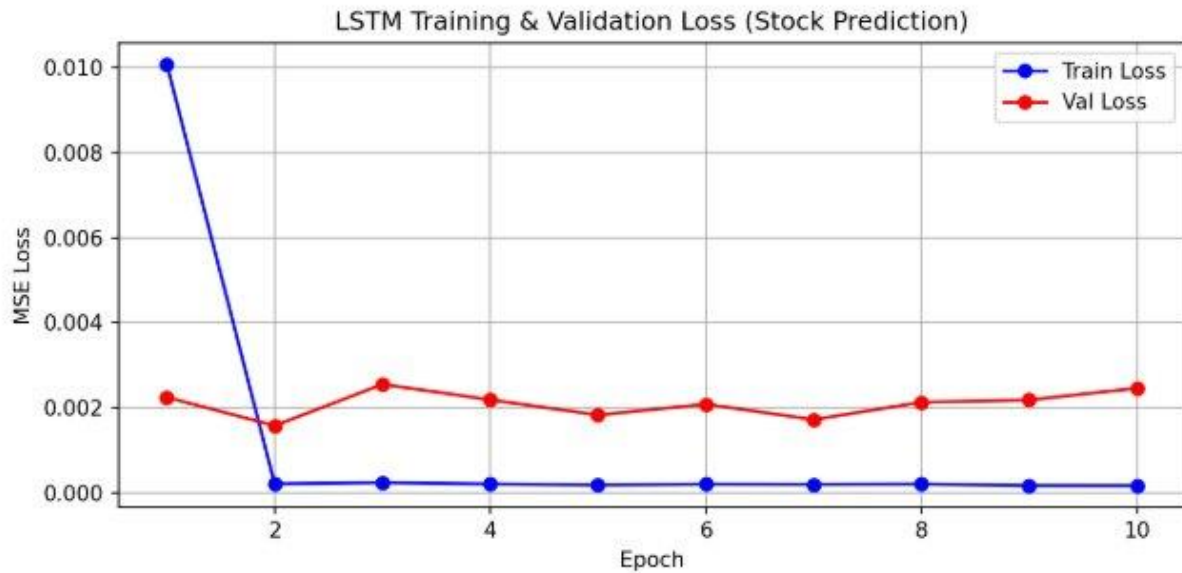
Using device: cuda
/tmp/ipykernel_9469/3495649871.py:20: FutureWarning: YF.download() has changed argument auto_adjust default to True
df = yf.download('GOOGL', start='2010-01-01', end='2024-12-01')
[*****100%*****] 1 of 1 completed
Price      Close      High      Low      Open      Volume
Ticker     GOOGL      GOOGL      GOOGL      GOOGL      GOOGL
Date
2010-01-04  15.555865  15.624369  15.493568  15.560829  78169752
2010-01-05  15.487363  15.582920  15.426555  15.566539  120067812
2010-01-06  15.096945  15.533775  15.049787  15.533775  158988852
2010-01-07  14.745496  15.140133  14.709508  15.125240  256315428
2010-01-08  14.942070  14.972598  14.621644  14.693374  188783028
Total data points: 3753
X_train shape: (2942, 60, 1), y_train shape: (2942, 1)
LSTMModel(
  (lstm): LSTM(1, 50, num_layers=2, batch_first=True, dropout=0.2)
  (fc1): Linear(in_features=50, out_features=25, bias=True)
  (fc2): Linear(in_features=25, out_features=1, bias=True)
  (relu): ReLU()
)
Epoch 1/10 | Train Loss: 0.010083 | Val Loss: 0.002245
Epoch 2/10 | Train Loss: 0.000206 | Val Loss: 0.001576
Epoch 3/10 | Train Loss: 0.000235 | Val Loss: 0.002548
Epoch 4/10 | Train Loss: 0.000202 | Val Loss: 0.002185
Epoch 5/10 | Train Loss: 0.000180 | Val Loss: 0.001826
Epoch 6/10 | Train Loss: 0.000198 | Val Loss: 0.002082
Epoch 7/10 | Train Loss: 0.000193 | Val Loss: 0.001718
Epoch 8/10 | Train Loss: 0.000200 | Val Loss: 0.002128
Epoch 9/10 | Train Loss: 0.000166 | Val Loss: 0.002180
Epoch 10/10 | Train Loss: 0.000164 | Val Loss: 0.002456

Predicted next day's price for GOOGL: $156.13

```

Training Loss Curve





Results and Analysis

1) Training Behavior

- Train loss dropped sharply from 0.010083 in Epoch 1 to 0.000164 by Epoch 10 — a ~98% reduction.
- Validation loss started at 0.002245 and fluctuated mildly, ending at 0.002456.
- The model converged quickly within the first 2 epochs and stabilized thereafter.
- Minor validation loss oscillation (0.0016–0.0026) indicates the model is near optimal but slightly sensitive to batch variation.

2) Price Prediction

- Predicted next day's price for GOOGL: \$156.13
- This prediction is derived from the last 60-day sequence of scaled price data.

TASK 2

Compare LSTM with Simple RNN

```
/tmp/ipykernel_9469/801625094.py:22: FutureWarning: YF.dc
df = yf.download('GOOGL', start='2010-01-01',
[*****100%*****] 1 of
```

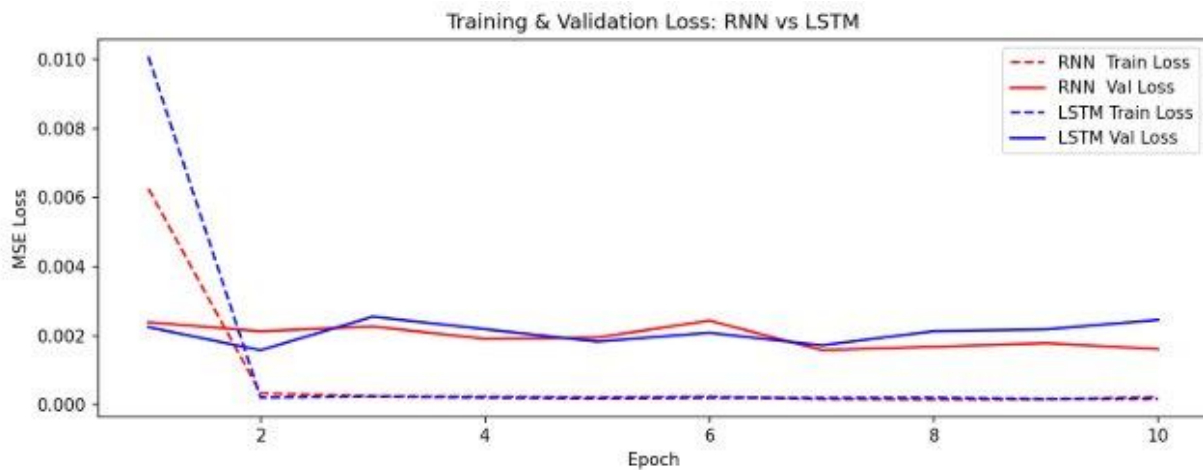
Training LSTM

```
Epoch 1/10 | Train: 0.010083 | Val: 0.002245
Epoch 2/10 | Train: 0.000206 | Val: 0.001576
Epoch 3/10 | Train: 0.000235 | Val: 0.002548
Epoch 4/10 | Train: 0.000202 | Val: 0.002185
Epoch 5/10 | Train: 0.000180 | Val: 0.001826
Epoch 6/10 | Train: 0.000198 | Val: 0.002082
Epoch 7/10 | Train: 0.000193 | Val: 0.001718
Epoch 8/10 | Train: 0.000200 | Val: 0.002128
Epoch 9/10 | Train: 0.000166 | Val: 0.002180
Epoch 10/10 | Train: 0.000164 | Val: 0.002456
```

Training Simple RNN

```
Epoch 1/10 | Train: 0.006257 | Val: 0.002387
Epoch 2/10 | Train: 0.000330 | Val: 0.002125
Epoch 3/10 | Train: 0.000243 | Val: 0.002264
Epoch 4/10 | Train: 0.000232 | Val: 0.001905
Epoch 5/10 | Train: 0.000211 | Val: 0.001944
Epoch 6/10 | Train: 0.000234 | Val: 0.002436
Epoch 7/10 | Train: 0.000159 | Val: 0.001579
Epoch 8/10 | Train: 0.000150 | Val: 0.001676
Epoch 9/10 | Train: 0.000153 | Val: 0.001781
Epoch 10/10 | Train: 0.000222 | Val: 0.001608
```

LSTM RMSE: 8.8713
RNN RMSE: 7.1782
Winner: Simple RNN





Results and Analysis

- Winner (by RMSE): Simple RNN, RMSE of 7.18 vs LSTM's 8.87.
- LSTM converged faster (low train loss from Epoch 2), while Simple RNN was slower to stabilize but achieved lower final validation loss (0.001608 vs 0.002456).
- The stock price prediction plot shows Simple RNN tracked actual prices more closely, confirming its lower RMSE on this task.
- This result is dataset-specific: on short 10-epoch runs with a single feature, the simpler RNN can outperform LSTM because daily stock prices do not always require LSTM's gating mechanisms.
- LSTM is generally expected to outperform on longer sequences, multi-feature inputs, and tasks with longer-range dependencies.

TASK 3

LSTM for Sentiment Analysis

Dataset

- IMDB Movie Reviews dataset , 25,000 training and 25,000 test reviews
- Binary classification: Positive (1) or Negative (0)
- Vocabulary size: 10,000 most frequent words, max sequence length: 200 tokens

Model Architecture

- Embedding Layer: 10,000 vocab $\rightarrow$ 64-dimensional embeddings
- LSTM: 2-layer stacked, 64 $\rightarrow$ 128 hidden units, Dropout(0.3)
- Dropout(0.4) before final classification layer
- Fully Connected: Linear(128 $\rightarrow$ 1) with Sigmoid activation

After padding - X_train: (25000, 200) | X_test: (25000, 200)

Model:

```
LSTMSentiment(  
  (embedding): Embedding(10000, 64, padding_idx=0)  
  (lstm): LSTM(64, 128, num_layers=2, batch_first=True, dropout=0.3)  
  (dropout): Dropout(p=0.4, inplace=False)  
  (fc): Linear(in_features=128, out_features=1, bias=True)  
)
```

Epoch 1/10		Train Loss: 0.6920	Acc: 51.53%		Val Loss: 0.6895	Acc: 52.70%
Epoch 2/10		Train Loss: 0.6850	Acc: 54.79%		Val Loss: 0.6847	Acc: 51.33%
Epoch 3/10		Train Loss: 0.6849	Acc: 54.76%		Val Loss: 0.6853	Acc: 56.10%
Epoch 4/10		Train Loss: 0.6836	Acc: 55.67%		Val Loss: 0.6814	Acc: 56.46%
Epoch 5/10		Train Loss: 0.6770	Acc: 57.76%		Val Loss: 0.6538	Acc: 63.97%
Epoch 6/10		Train Loss: 0.6244	Acc: 66.98%		Val Loss: 0.6055	Acc: 69.28%
Epoch 7/10		Train Loss: 0.5165	Acc: 77.17%		Val Loss: 0.5755	Acc: 73.16%
Epoch 8/10		Train Loss: 0.4429	Acc: 80.98%		Val Loss: 0.4641	Acc: 78.99%
Epoch 9/10		Train Loss: 0.3974	Acc: 83.43%		Val Loss: 0.4481	Acc: 79.96%
Epoch 10/10		Train Loss: 0.3474	Acc: 86.14%		Val Loss: 0.4062	Acc: 82.13%

Final Test Accuracy: 82.13%

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ————— 1s 1us/step

--- Custom Review Predictions ---

Review : 'This movie was absolutely brilliant I loved every moment'
Result : POSITIVE (confidence: 0.90)

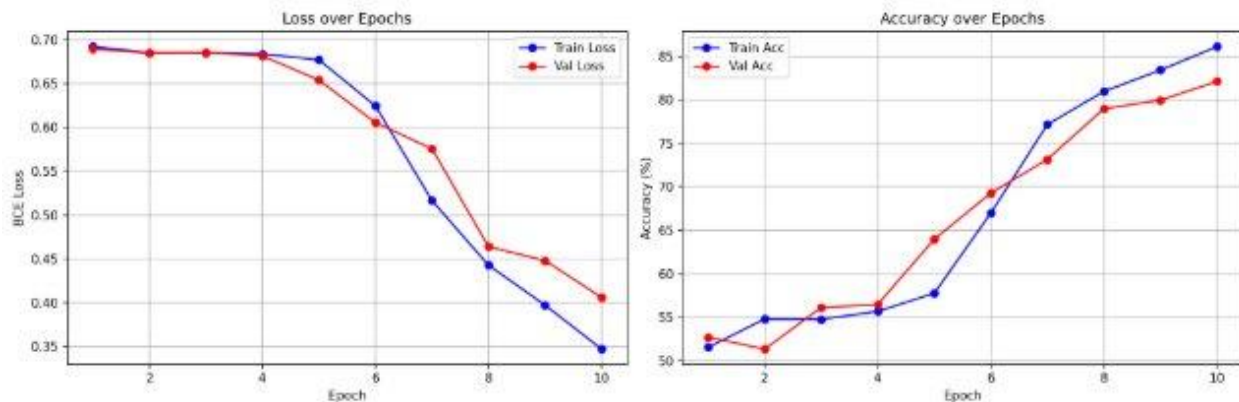
Review : 'Terrible film complete waste of time boring and awful'
Result : NEGATIVE (confidence: 0.05)

Review : 'One of the best films I have seen in years highly recommend'
Result : POSITIVE (confidence: 0.92)

Review : 'I walked out halfway through it was that bad'
Result : NEGATIVE (confidence: 0.26)

Review : 'The acting was decent but the story felt a bit slow'
Result : NEGATIVE (confidence: 0.46)

Training Results



Analysis

- Final Test Accuracy: 82.13% after 10 epochs.
- Accuracy improved slowly for the first 4 epochs (~52–56%) then accelerated sharply from Epoch 5 onward, typical behavior as the model learns meaningful embeddings before LSTM can leverage sequential context.
- Both train and validation loss decreased consistently with no divergence, confirming no overfitting.
- Strong positive reviews ('brilliant', 'highly recommend') were classified with high confidence (0.90–0.92).
- Clear negative reviews ('terrible', 'waste of time') were correctly identified with confidence near 0.
- Ambiguous review ('acting was decent but story felt slow') was classified as negative (0.46), showing the model captures nuance.

Conclusion

The LSTM model successfully predicted GOOGL stock prices with a next-day forecast of \$156.13 and demonstrated fast convergence in training. When compared with Simple RNN over 10 epochs, the Simple RNN achieved a lower RMSE (7.18 vs 8.87), showing that for short training runs on single-feature financial time series, a simpler model can be competitive. However, LSTM's architectural advantages, gating mechanisms that control information flow, make it more robust for complex, long-range temporal patterns.

For sentiment analysis on IMDB reviews, the LSTM model reached 82.13% test accuracy, correctly classifying both strongly positive and strongly negative reviews with high confidence, and handling ambiguous language reasonably well. Further improvements could be achieved by using pre-trained word embeddings (GloVe, Word2Vec), bidirectional LSTM layers, or transformer-based architectures.